STUDENT MANAGEMENT SYSTEM

DETAILED DBMS MINI PROJECT REPORT

Database: student_management

Technology: MySQL / MySQL Workbench

Submitted by: _____

Register Number: _____

Course / Semester: _____

Department: _____

College: _____

Submitted to: _____

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the mini project report entitled **“Student Management System”** has been prepared as part of the academic requirements for the DBMS course. The report describes a relational database implemented using MySQL, including the design of tables, primary and foreign keys, sample records, SQL queries, and generated reports.

Faculty Signature: _____

Head of Department: _____

Date: _____ Place: _____

DECLARATION

I hereby declare that this project report on the Student Management System has been prepared for academic learning and demonstration. The SQL schema and sample data are used to explain how student information can be organized and queried in a relational database. Any personal details shown in the sample records are illustrative project data.

Student Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my subject faculty and department for the guidance and encouragement provided during the preparation of this DBMS mini project. I also thank my institution for providing the opportunity to learn database design and SQL implementation through a practical project.

This project helped me understand the importance of structured data storage, table relationships, integrity constraints, and SQL queries. I am thankful to everyone who supported the completion of this report.

ABSTRACT

The Student Management System is a database application concept designed to organize student-related academic and administrative information. The supplied SQL script creates a database named **student_management** and six related tables: courses, students, subjects, marks, attendance, and fees. It inserts sample records and demonstrates basic data retrieval, joins, aggregate functions, sorting, conditional filtering, subqueries, and a view.

The database links students to courses, subjects to courses, marks to both students and subjects, and attendance and fees to students. Reports can be generated to show student results, subject grades, attendance percentages, fee balances, and overall totals. The project demonstrates how a relational database can improve consistency and make common administrative queries easier.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Scope of the Project

5. Hardware and Software Requirements

6. System Analysis

7. Database Design

8. ER Diagram and Relationships

9. Table Structure

10. Implementation Procedure

11. SQL Queries and MySQL Outputs

12. Academic Result and Grade Reports

13. Attendance and Fee Reports

14. Testing and Validation

15. Advantages and Limitations

16. Future Enhancements

17. Conclusion

18. References

1. INTRODUCTION

A Student Management System stores and organizes information required to manage students in an educational institution. In a manual system, student records, marks, attendance, and fee information may be maintained in separate registers or spreadsheets. Searching and combining this information can take time and may lead to duplication or mistakes.

A relational database provides a structured method for storing this information. In this project, MySQL is used to create six related tables. Primary keys uniquely identify records, while foreign keys connect related information. SQL queries retrieve information and produce useful reports.

1.1 Purpose

The purpose of the project is to demonstrate the use of a relational database for student administration. It is a learning project and does not include a full graphical application or user-login system.

1.2 Need for the System

• Store student details in a consistent format.

• Connect each student to a course.

• Maintain course-wise subjects and marks.

• Record daily attendance.

• Track fee amounts paid and pending.

• Generate reports using SQL.

2. PROBLEM STATEMENT

Educational institutions need to maintain accurate student information, including admission details, course enrolment, subjects, marks, attendance, and fees. When these details are stored separately, it can be difficult to produce combined reports. The project addresses this need by organizing the information into related database tables and using SQL to retrieve it.

3. OBJECTIVES

• Create a database named student_management.

• Create tables with suitable data types and constraints.

• Store sample course, student, subject, marks, attendance, and fees records.

• Use primary keys and foreign keys to define relationships.

• Retrieve student and course details using JOIN operations.

• Calculate average marks, total marks, highest marks, and lowest marks.

• Calculate attendance percentages and identify low-attendance students.

• Calculate pending fee balances and total fee collection.

• Create a view for a reusable student result summary.

4. SCOPE OF THE PROJECT

The current project covers database creation, table creation, sample data insertion, data retrieval, and report generation. It includes course-wise student counts, marks summaries, grades, attendance percentages, fee balances, and a student report view.

The current SQL script does not include a web interface, login and role management, automated notifications, online payment integration, or backups. These can be considered as future enhancements.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

• Computer or laptop capable of running MySQL.

• Sufficient RAM and disk space for MySQL and project files.

• Keyboard, display, and storage device.

5.2 Software Requirements

Item	Description
Database server	MySQL Server
SQL client	MySQL Workbench or MySQL command-line client
Operating system	Windows, Linux, or macOS
Project language	SQL
Input file	Student_Management_System_MySQL SQL script

6. SYSTEM ANALYSIS

6.1 Existing Approach

A manual or disconnected-file approach can require repeated entry of the same information. It may also be difficult to link a student's personal details with academic results, attendance, and fee balances.

6.2 Proposed Approach

The proposed approach uses a relational MySQL database. Each category of information is stored in a dedicated table. The tables are linked through key fields, allowing related information to be combined when a report is requested.

6.3 Main Users

• Administrator or office staff: maintain student, course, and fee details.

• Faculty: review subjects, marks, and attendance records.

• Student: in a future application, view personal academic information.

7. DATABASE DESIGN

Database name: **student_management**. The script first drops an existing database with the same name, creates a fresh database, and selects it for the remaining operations. **Warning:** running the supplied script again deletes the existing student_management database and its data before recreating it.

7.1 Design Principles

• Use a primary key to identify each row uniquely.

• Use foreign keys to connect related tables.

• Use NOT NULL for required values.

• Use UNIQUE for the student email and for the student-subject-exam combination in marks.

• Use ENUM for gender and attendance status where defined.

• Use DECIMAL for fee amounts to preserve decimal currency values.

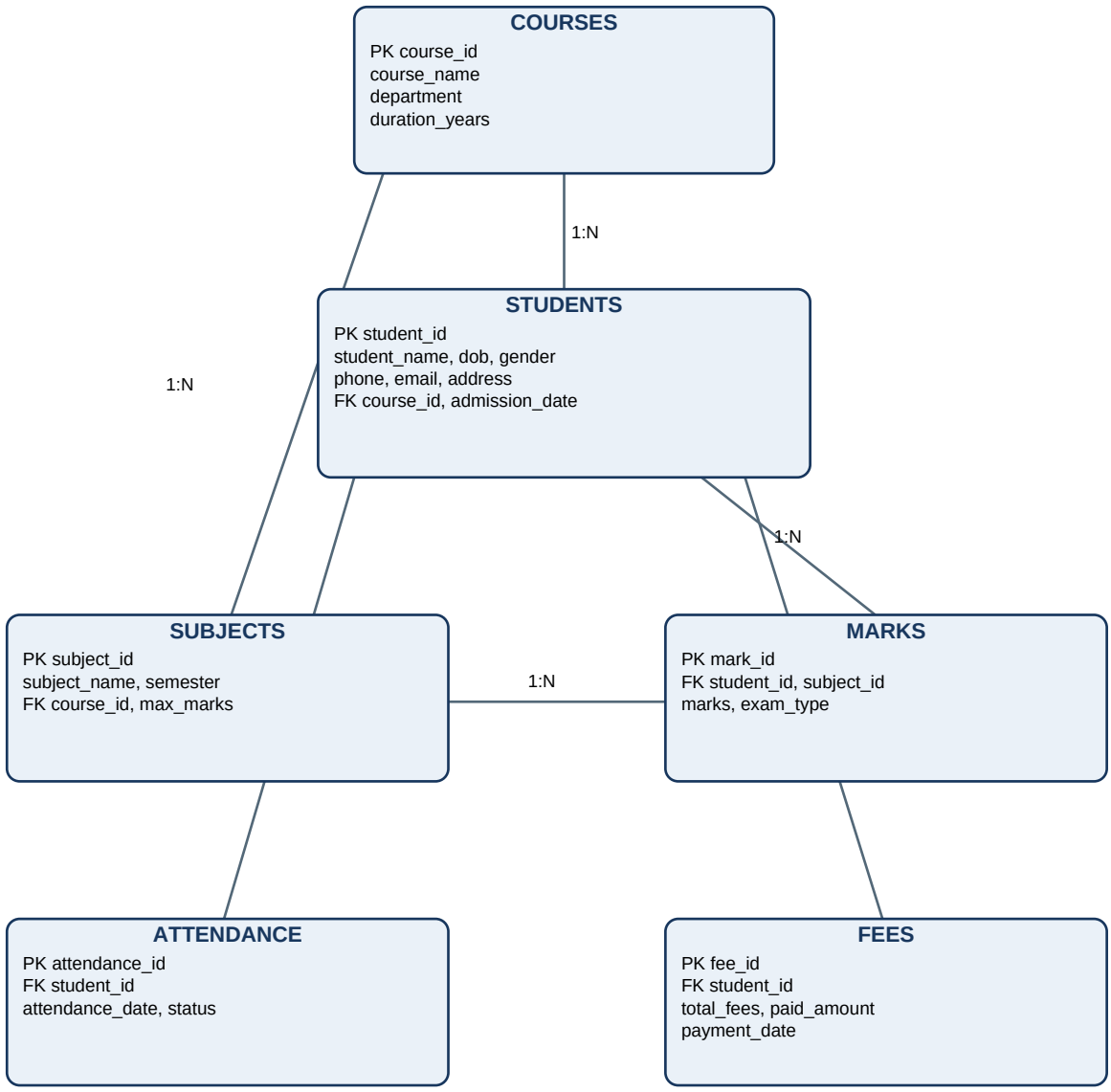
7.2 Table Summary

Table	Primary Key	Purpose
courses	course_id	Course name, department, duration
students	student_id	Personal and admission details; course link
subjects	subject_id	Subject name, course link, semester, maximum marks

Table	Primary Key	Purpose
marks	mark_id	Student/subject marks and exam type
attendance	attendance_id	Date-wise attendance status
fees	fee_id	Total fee, paid amount, payment date

8. ER DIAGRAM AND RELATIONSHIPS

The ER diagram shows the main entities and their relationships. PK means Primary Key and FK means Foreign Key.



8.1 Relationship Explanation

• One course can be linked to many students through students.course_id.

• One course can contain many subjects through subjects.course_id.

• One student can have many marks records through marks.student_id.

• One subject can appear in many marks records through marks.subject_id.

• One student can have many attendance records through attendance.student_id.

• One student can have fee records through fees.student_id.

9. TABLE STRUCTURE

9.1 Courses

Fields: course_id (INT, primary key, auto-increment), course_name (VARCHAR(100), required), department (VARCHAR(100), required), duration_years (INT, required).

9.2 Students

Fields: student_id (INT, primary key, auto-increment), student_name (VARCHAR(100), required), dob (DATE), gender (ENUM: Male/Female/Other), phone (VARCHAR(15)), email (VARCHAR(100), unique), address (VARCHAR(200)), course_id (INT, required foreign key), admission_date (DATE, required).

9.3 Subjects

Fields: subject_id (INT, primary key, auto-increment), subject_name (VARCHAR(100), required), course_id (INT, required foreign key), semester (INT, required), max_marks (INT, default 100).

9.4 Marks

Fields: mark_id (INT, primary key, auto-increment), student_id (INT, required foreign key), subject_id (INT, required foreign key), marks (INT, required), exam_type (VARCHAR(50), default Internal). The combination student_id, subject_id, and exam_type must be unique.

9.5 Attendance

Fields: attendance_id (INT, primary key, auto-increment), student_id (INT, required foreign key), attendance_date (DATE, required), status (ENUM: Present/Absent, required).

9.6 Fees

Fields: fee_id (INT, primary key, auto-increment), student_id (INT, required foreign key), total_fees (DECIMAL(10,2), required), paid_amount (DECIMAL(10,2), default 0), payment_date (DATE).

10. IMPLEMENTATION PROCEDURE

1. Install and start MySQL Server.

2. Open MySQL Workbench and connect to the server.

3. Open the supplied SQL script in a query tab.

4. Review the DROP DATABASE statement before running the script, because it resets the database.

5. Execute the script to create the database and tables and insert sample records.

6. Run SHOW DATABASES; and USE student_management; to select the database.

7. Run SHOW TABLES; to verify that the six tables exist.

8. Use DESC table_name; to inspect each table structure.

9. Run the SELECT, JOIN, aggregate, attendance, fee, and result queries.

10. Check each result grid and capture screenshots if your teacher requires actual MySQL output.

10.1 Basic Commands

```
SHOW DATABASES;
USE student_management;
SHOW TABLES;
DESC students;
SELECT * FROM students;
```

11. SQL QUERIES AND MYSQL OUTPUTS

The following are sample output tables calculated from the provided SQL data. They are report-style examples rather than screenshots from a running MySQL session.

11.1 SHOW TABLES

Tables_in_student_management
attendance
courses
fees
marks
students
subjects

11.2 Total students

total_students
8

11.3 Students in each course

course_name	total_students
Computer Science	3
Information Science	2
Electronics and Communication	2
Mechanical Engineering	1

11.4 Highest and lowest marks

highest_mark	lowest_mark
94	72

12. ACADEMIC RESULT AND GRADE REPORTS

12.1 Student Result Summary

Average and total marks

student_name	course_name	average_marks	total_marks	subjects_count
Priya R	Information Science	89.67	269	3
Anjali P	Computer Science	88.50	354	4
Kiran S	Electronics and Communication	88.50	177	2
Arun Kumar	Computer Science	85.50	342	4
Vijay M	Electronics and Communication	81.00	162	2
Sneha Gowda	Information Science	79.00	237	3

student_name	course_name	average_marks	total_marks	subjects_count
Rahul Sharma	Computer Science	77.00	308	4
Manoj Kumar	Mechanical Engineering	77.00	154	2

12.2 Grade Rules

Grade calculation

Marks range	Grade
90 and above	A+
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	F

The grade report uses a CASE expression. It checks marks from the highest threshold to the lowest and assigns the corresponding grade.

SQL: Grade Report

```
SELECT s.student_name, sub.subject_name, m.marks,
CASE WHEN m.marks >= 90 THEN 'A+'
WHEN m.marks >= 80 THEN 'A'
WHEN m.marks >= 70 THEN 'B'
WHEN m.marks >= 60 THEN 'C'
WHEN m.marks >= 50 THEN 'D'
ELSE 'F' END AS grade
FROM marks m
JOIN students s ON m.student_id = s.student_id
JOIN subjects sub ON m.subject_id = sub.subject_id
ORDER BY s.student_name;
```

13. ATTENDANCE AND FEE REPORTS

13.1 Attendance Percentage

Attendance summary

student_name	total_classes	present_classes	attendance_percentage
Arun Kumar	4	3	75.00%
Rahul Sharma	4	3	75.00%
Priya R	4	4	100.00%
Sneha Gowda	4	3	75.00%
Kiran S	4	3	75.00%
Manoj Kumar	4	3	75.00%
Anjali P	4	4	100.00%
Vijay M	4	3	75.00%

Attendance percentage is calculated as present classes divided by total recorded classes, multiplied by 100. In the sample data, two students have 100% attendance and the remaining six have 75%. The query for students below 75% returns no rows for these sample records.

13.2 Fee Balance

Fee report

student_name	total_fees	paid_amount	balance
Arun Kumar	60000.00	60000.00	0.00
Rahul Sharma	60000.00	45000.00	15000.00
Priya R	55000.00	55000.00	0.00
Sneha Gowda	55000.00	40000.00	15000.00
Kiran S	65000.00	50000.00	15000.00
Manoj Kumar	65000.00	65000.00	0.00
Anjali P	60000.00	30000.00	30000.00
Vijay M	65000.00	45000.00	20000.00

Fee totals

total_fees_collected	total_pending_fees
400000.00	95000.00

The balance is calculated by subtracting paid_amount from total_fees. The sample data shows total fee collection of 400,000 and a pending balance of 95,000.

14. TESTING AND VALIDATION

Test	Operation	Expected result
T1	Create database	student_management database is created
T2	SHOW TABLES	Six project tables are listed
T3	Count students	Returns 8
T4	Join students and courses	Each student is shown with course information
T5	Aggregate marks	Returns averages, totals, maximum and minimum
T6	Attendance report	Returns attendance counts and percentages
T7	Fee report	Returns paid amount and remaining balance
T8	Student report view	View returns grouped student results

These are logical test cases based on the SQL statements. Actual execution should be performed in MySQL Workbench to verify the server output and capture genuine screenshots.

15. ADVANTAGES AND LIMITATIONS

15.1 Advantages

• Related data is stored in separate, organized tables.

• Primary and foreign keys support consistency.

• JOIN queries combine information from multiple tables.

• Aggregate queries simplify academic and financial summaries.

• The student_report view makes result summaries reusable.

15.2 Limitations

• The current project is SQL-based and has no graphical user interface.

• The sample data is limited and intended for demonstration.

• The script does not implement login, permissions, or automated backups.

• The fees table permits multiple rows per student; the sample contains one row per student.

16. FUTURE ENHANCEMENTS

• Develop a web or desktop user interface for administrators and faculty.

• Add secure login and role-based access control.

• Add student profile updates and course enrolment workflows.

• Add examination schedules, results publishing, and downloadable mark sheets.

• Add fee receipt generation and payment-history tracking.

• Add automated attendance summaries and notifications.

• Introduce database backup, recovery, and audit logging.

17. CONCLUSION

The Student Management System demonstrates the design and use of a relational database with MySQL. It organizes courses, students, subjects, marks, attendance, and fees into six connected tables. SQL operations are used to retrieve records, join related data, calculate academic summaries, generate grades, measure attendance, and report fee balances.

The project meets its learning objectives by demonstrating table creation, constraints, data insertion, joins, aggregate functions, subqueries, ordering, conditional filtering, and a database view. It can be extended into a larger application with a user interface and additional security features.

18. REFERENCES

• Supplied SQL file: Student_Management_System_MySQL-2.sql.txt.

• MySQL documentation: https://dev.mysql.com/doc/

• MySQL Workbench documentation: https://dev.mysql.com/doc/workbench/en/

Note: The report's sample output tables are derived from the supplied sample data and query logic. They are formatted examples, not screenshots captured from a live MySQL console.